API & Authentication

Name: Noora Najeeb Altamimi

API stands for Application Programming Interface. APIs are mechanisms that enable two software components to communicate with each other using a set of definitions and protocols. For example, the weather bureau's software system contains daily weather data. The weather app on your phone "talks" to this system via APIs and shows you daily weather updates on your phone. In the context of APIs, the word Application refers to any software with a distinct function. Interface can be thought of as a contract of service between two applications. This contract defines how the two communicate with each other using requests and responses. Their API documentation contains information on how developers are to structure those requests and responses.

Why APIs exists?

REST APIs offer four main benefits:

1. Integration

APIs are used to integrate new applications with existing software systems. This increases development speed because each functionality doesn't have to be written from scratch. You can use APIs to leverage existing code.

2. Innovation

Entire industries can change with the arrival of a new app. Businesses need to respond quickly and support the rapid deployment of innovative services. They can do this by making changes at the API level without having to re-write the whole code.

3. Expansion

APIs present a unique opportunity for businesses to meet their clients' needs across different platforms. For example, maps API allows map information integration via websites, Android,iOS, etc. Any business can give similar access to their internal databases by using free or paid APIs.

4. Ease of maintenance

The API acts as a gateway between two systems. Each system is obliged to make internal changes so that the API is not impacted. This way, any future code changes by one party do not impact the other party

What is REST?

REST (Representational State Transfer) is an architectural style used to design APIs and enable communication between client and server systems over the internet.

REST APIs are designed to be:

- simple
- scalable
- reliable
- easy to maintain

APIs that follow REST architecture are called REST APIs or RESTful APIs.

Main Principles of REST

1. Uniform Interface

REST APIs use a standard way for communication between clients and servers.

Resources are identified using URLs.

Example:

https://api.example.com/users

This consistency makes APIs easier to understand and use.

2. Statelessness

Each request in REST is independent.

The server does not store information about previous requests.

This means every request must contain all the information needed to process it.

3. Layered System

REST architecture can use multiple layers such as:

- security
- application logic
- database servers

The client does not need to know how these layers work internally.

4. Cacheability

REST APIs support caching to improve performance and reduce server load.

Frequently used data can be temporarily stored and reused instead of requesting it repeatedly from the server.

5. Code on Demand

In some cases, the server can send executable code to the client to extend functionality temporarily.

For example:

- form validation in websites

Frontend and Backend Communication

What Are Frontend and Backend?

Frontend

The frontend is the part of the application that users see and interact with directly including buttons, forms, pages. . .

Backend

The backend is the hidden part of the application responsible for processing data and handling business logic. It manages databases, authentication, server logic. . .

Because Frontend and backend are separate systems (The frontend cannot directly access the database or server logic safely). Therefore, a communication method is needed between them. So API's role come now to solve this problem an API acts as a bridge between the frontend and backend. It allows applications to communicate by sending requests and receiving responses in an organized and secure way.

How Frontend and Backend Communicate

1- User Action:

The communication process starts when the user performs an action for example clicking login button.

2- Frontend Sends Request:
The frontend sends an HTTP request to the backend through an API called request because the frontend is requesting a service or information from the backend.
The request may contain login data such as: email and password.

3- Backend Processes the Request:
The backend receives the request and processes it for example it may check database or validate login information. . .

4- Backend Sends a Response:
After processing the request, the backend sends a response back to the frontend.
Successful response example:

```
{
  “Status”:”success”
}
```

Error response example:

```
{
  "status": "error"
}
```

5- Frontend Displays the Result:
Finally, the frontend displays the result to the user maybe open the home page after successful login or show an error message if the password is incorrect.

Real-Life Example

A weather application works using frontend-backend communication.

- The frontend displays the weather interface.
- The backend retrieves weather data from the server.
- The API transfers the data between them.

For example:

- the frontend requests the weather of a city
- the backend processes the request
- the response returns the temperature and weather information

REST & Endpoints:

1- GET:
The GET HTTP method is used to request and retrieve data from a server. It is mainly used to read information and should not modify data on the server.
Examples of using GET:

- retrieving user information
- loading a webpage
- fetching products from a database

GET requests usually send information through the URL using query parameters.
Example:
GET /contact HTTP/1.1
Host: example.com

If the request is successful, the server returns the requested data with a success response such as:
HTTP/1.1 200 OK

2- POST:
The POST HTTP method is used to send data to the server. It is commonly used to create new resources such as user accounts, orders, or form submissions.
Examples of using POST:

- creating a new account
- submitting a form
- adding new data to a database
  POST requests usually change data on the server.
  The difference between PUT and POST is that PUT is idempotent: calling it once is no different from calling it several times successively (there are no *side* effects). Successive identical POST requests may have additional effects, such as creating the same order several times.
- pressing “Create Order” twice using POST may create two orders.
- updating the same profile using PUT keeps the same result.

3- PUT:
The **PUT** HTTP method creates a new resource or replaces a representation of the target resource with the request content.
The difference between PUT and POST is that PUT is idempotent: calling it once is no different from calling it several times successively (there are no *side* effects).
Examples:
Examples of successfully creating resources:
The following PUT request asks to create a resource at example.com/new.html with the content New File :
PUT /new.html HTTP/1.1
Host: example.com
Content-type: text/html
Content-length: 16

New File

If the target resource **does not** have a current representation and the PUT request successfully creates one, then the origin server must send a 201 Created response:
HTTP/1.1 201 Created
Content-Location: /new.html

If the target resource **does** have a current representation and that representation is successfully modified with the state in the request, the origin server must send either a 200 OK or a 204 No Content to indicate successful completion of the request:
HTTP/1.1 204 No Content
Content-Location: /existing.html

4- DELETE:

The **DELETE** HTTP method asks the server to delete a specified resource.
Requests using DELETE should only be used to delete data and shouldn't contain a body.
Examples:
Successfully deleting a resource:
The following request asks the server to delete the resource file.html:
DELETE /file.html HTTP/1.1
Host: example.com
If the request is successful, there are several possible successful response status codes. A 204 No Content response means the request was successful and no additional information needs to be sent back to the client:
HTTP/1.1 204 No Content
Date: Wed, 04 Sep 2024 10:16:04 GMT
A 200 OK response means the request was successful and the response body includes a representation describing the outcome:
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Date: Fri, 21 Jun 2024 14:18:33 GMT
Content-Length: 1234

```
<html lang="en-US">
  <body>
    <h1>File "file.html" deleted.</h1>
  </body>
</html>
```

A 202 Accepted response means the request has been accepted and will probably succeed, but the resource has not yet been deleted by the server.

```
HTTP/1.1 202 Accepted
Date: Wed, 26 Jun 2024 12:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 1234

<html lang="en-US">
  <body>
    <h1>Deletion of "file.html" accepted.</h1>
    <p>See <a href="http://example.com/tasks/123/status">the status monitor</a> for
details.</p>
  </body>
</html>
```

- **What are Authentication & Authorization?**

authentication is the process of verifying who a user is, while authorization is the process of verifying what they have access to. Comparing these processes to a real-world example, when you go through security in an airport, you show your ID to authenticate your identity. Then, when you arrive at the gate, you present your boarding pass to the flight attendant, so they can authorize you to board your flight and allow access to the plane.

- **Authentication**
  Determines whether users are who they claim to be, Challenges the user to validate credentials (for example, through passwords, answers to security questions, or facial recognition), Usually done before authorization, Generally, transmits info through an ID Token,
- **Authorization**
  Determines what users can and cannot access, Verifies whether access is allowed through policies and rules, Usually done after successful authentication, Generally, transmits info through an Access Token, Example: After an employee successfully authenticates, the system determines what information the employees are allowed to access.

HTTP authentication:

HTTP provides a general framework for access control and authentication.

The general http framework:

defines the HTTP authentication framework, which can be used by a server to challenge a client request, and by a client to provide authentication information.

❖ The challenge and response flow works like this:

1. The server responds to a client with a 401 (Unauthorized) response status and provides information on how to authorize with a WWW-Authenticate response header containing at least one challenge.
2. A client that wants to authenticate itself with the server can then do so by including an Authorization request header with the credentials.
3. Usually a client will present a password prompt to the user and will then issue the request including the correct Authorization header.

Basic authentication:

Basic Authentication is a simple authentication method where the client sends a username and password with each request to verify identity.

Security of Basic Authentication:

As the user ID and password are passed over the network as clear text, the basic authentication scheme is not secure. HTTPS/TLS should be used with basic authentication to prevent credential interception.

❖ Access using credentials in the URL

Historically some sites would allow you to login using an encoded URL containing the username and the password as shown:

https://username:password@www.example.com/

This syntax is no longer allowed in modern browsers; the username and password are stripped from the request before it is sent.

- **JWT (JSON Web Tokens)** :
  is an open standard that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. Because of its relatively small size, a JWT can be sent through a URL, through a POST parameter, or inside an HTTP header, and it is transmitted quickly. A JWT contains all the required information about an entity to avoid querying a database more than once. The recipient of a JWT also does not need to call a server to validate the token.

Benefits:

- **Compact**: JWTs are small in size, which makes them a good choice to be passed in HTML and HTTP environments.

- **Secure**: JWTs can use a public/private key pair in the form of an X.509 certificate for signing.
- **Common**: JSON parsers are supported by most programming languages.

Usage:

- **Authentication**: When a user successfully logs in using their credentials, an ID token is returned. According to the OpenID Connect (OIDC) specification, an **ID token is always a JWT.**
- **Authorization**: Once a user is successfully logged in, an application may request to access routes, services, or resources (for example, APIs) on behalf of that user. To do so, the application must pass an Access Token in every request, which may be in the form of a JWT.
- **Information exchange**: JWTs are a good way of securely transmitting information between parties because they can be signed, which means you can be certain that the senders are who they say they are. Additionally, the structure of a JWT allows you to verify that the content hasn't been tampered with.

❖ How Login Works Under the Hood

- The user enters username and password.
- The frontend sends the credentials to the backend.
- The backend verifies the information.
- If valid, the server generates a token (such as JWT).
- The frontend stores the token.
- The token is sent with future requests to prove the user identity.

❖ **Why We Never Store Plain Text Passwords:**
Storing passwords as plain text is extremely dangerous because anyone who gains access to the database can immediately read all user passwords. This may

lead to:

- account theft
- data breaches
- unauthorized access to sensitive information

For this reason, modern applications use password hashing instead of storing the actual passwords.

How Password Hashing Works:
When a user creates a password, the system applies a hash function to convert the password into a fixed hash value before storing it in the database.

During login:

1. The user enters the password.
2. The password is hashed again using the same hashing algorithm.
3. The generated hash is compared with the stored hash.
4. If both hashes match, the user is authenticated successfully.

This process ensures that the real password is never stored directly in the database.

Example:
Plain Password:

mypassword123

Hashed Value Example:

5f4dcc3b5aa765d61d8327deb882cf99

Benefits of Password Hashing:

- Improves security
- Protects user credentials
- Reduces risks during database leaks
- Prevents attackers from reading real passwords directly